

The Mathematics behind the Linear Regression

Gavin Simpson

May 31, 2026

1 Overview

What is a linear regression? essentially with a linear regression, we are aiming to find a linear relationship between two variables (2 Dimensional for now) and fit a line through the data points with the best accuracy possible - now with this line we can interpolate and extrapolate to our needs.

1.1 Equation of the Regression Line

As this is a linear regression, the equation of the best fit line is going to be a simple linear function

$$y = mx + c \tag{1}$$

where:

- **m** (β_1) is the gradient
- **c** (β_0) is the bias

In this Regression line, there are *two* **parameters**, (β_0) and (β_1). The algorithm searches for the line of best fit by tweaking these two parameters until a line of best fit is found.

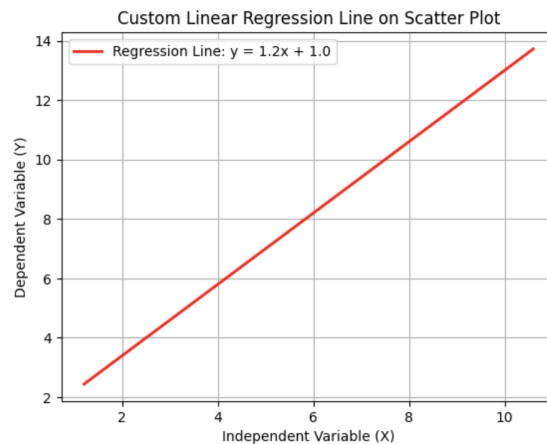


Figure 1: Visualised linear function

1.2 Initial Regression Line

When the linear regression algorithm starts, the parameters are given initial values - these tend to be starting points for the algorithm to work with, suitable values exemplify:

- $\beta_1 = 2$
- $\beta_0 = 1$

1.3 Evaluating a Regression Line

To evaluate our regression line - we use a **Loss function**, one of the most common ones is the **MSE** (Mean squared error).

1.3.1 MSE

To calculate the MSE, we simply sum the squares of differences between our predicted value and the observed value (dataset value). The formula is

$$\text{MSE}(\text{parameters}) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

where:

- $\hat{y}_i$ is the Predicted Value $f(x_i)$
- y_i is the Observed Value (from our Dataset)
- Summation: Represents going through all the values of the dataset

This loss function could be expanded as:

$$\text{MSE}(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^2$$

where:

- $\hat{y}_i$ has been expanded as $\beta_1 x_i + \beta_0$

2 Parameter Optimisation

The task here is to find the two parameters (β_0, β_1) , which will give us the lowest MSE. One way to do this is to perform the gradient descent in order to find the minimum parameters that lead to the lowest value on our MSE Loss function. How does this concept work? We can start by visualising our Loss landscape of the MSE.

2.1 Visualising our Loss Landscape

If we plot on a 3D axis

- β_0 ,
- β_1

- MSE

We will get the landscape of our loss function - with this, we can see how different values of β_0 (b), β_1 (m) will impact our MSE. Here is a simple example of what the MSE could look like

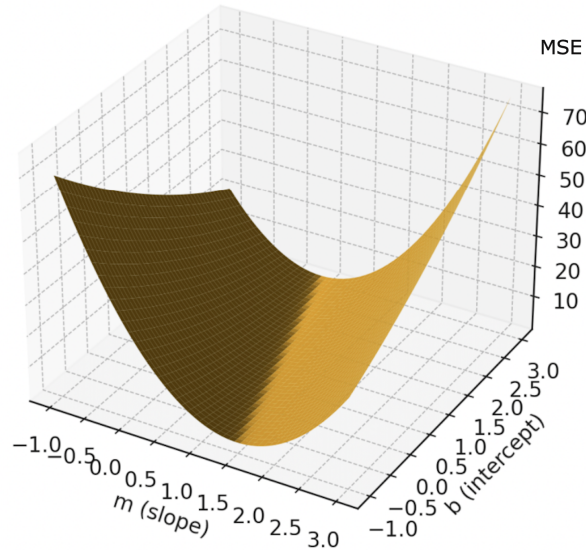


Figure 2: Loss Landscape of the MSE ($b = \beta_0$, $m = \beta_1$)

The Loss landscape is a plane; with this - we wish to find the minimum points/pits of this loss landscape - it is easier to conceptualize upon visualization.

3 The gradient Descent

The gradient descent is an iterative mathematic method used to find the minimum point of a function by going in steps to the opposite direction of the gradient until we hit a minimum point. An analogy to conceptualize this is to *imagine dropping a ball onto a surface, the ball will keep rolling until it rests at the lowest point of the surface - similar idea here.*

Right away we can see one main issue with the gradient descent - **local minimas**. If the surface has many local minimum points along with the *global minima*, the algorithm might get trapped and converge into a local minima instead of the global minima, hence making sure the starting points are randomized is a work around for this.

3.1 The gradient Descent Exemplified with a Quadratic Function

Assume we have a linear quadratic function and we wish to find it's minimum point VIA the gradient descent. Here is how the algorithm will work.

1. Start at a starting point - the intial X value
2. The gradient will tell you the direction of the minimum point - if there's a negative gradient, that means the curve is sloping downwards towards the minimum, if there's a positive gradient, that means the curve is sloping upwards away from the minimum.

3. Hence, our **steps** must be in the **opposite direction** of the gradient.
4. How big must our steps be? Well there are two neat tricks for this
 - (a) At the minimum point of the function, the curve has a gradient of 0

$$\frac{dy}{dx} = 0$$

and as the function approaches the minimum point, the gradient too tends to 0, this naturally helps us to converge the step length as we approach the minimum.

- (b) The learning rate: (α), on our iteration formula, we add a coefficient, the alpha, to control the step rate so that
5. Hence with these in mind, the iteration formula will be:

$$x = x - \alpha \cdot \left. \frac{dy}{dx} \right|_{x=x}$$

4 The Gradient Descent on a Bivariate Loss Function (MSE)

We are going to apply the same concept on our Loss function

$$\text{MSE}(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^2$$

But this time, its bivariate (a function with 2 variables), hence if we are going to use the gradient descent, we'll have to use **Partial Derivatives**

4.1 What are Partial Derivatives?

A partial derivative simply measures the rate of change of the whole function, with respects to just one variable whilst keeping the others constant. With our MSE:

- We simply take a constant β_0 , and find the value of β_1 that gives the lowest MSE for that β_0
- and then we find the β_0 that gives use the lowest MSE for that β_1

Hence with the iteration step, there will be two iterations conducted, one with respects to β_0 and another with respects to β_1

$$\beta_0 = \beta_0 - \alpha \cdot \left. \frac{\partial}{\partial \beta_0} \right|_{\beta_0, \beta_1}$$

$$\beta_1 = \beta_1 - \alpha \cdot \left. \frac{\partial}{\partial \beta_1} \right|_{\beta_0, \beta_1}$$

4.2 Computing the Partial Derivatives

we can use methods of calculus such as simply chain rules to differentiate them - whilst treating the other variable simply as a constant.

$$\text{MSE}(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^2$$

$$\left. \frac{\partial}{\partial \beta_0} \right|_{\beta_0, \beta_1} = \frac{2}{n} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^1 \cdot 1$$

$$\left. \frac{\partial}{\partial \beta_1} \right|_{\beta_0, \beta_1} = \frac{2}{n} \sum_{i=1}^n (\beta_1 x_i + \beta_0 - y_i)^1 \cdot x_i$$

With this, we can use an iterative formula to find the parameters (β_0, β_1) that give the lowest MSE.

5 Further Resources

I've done further work on Polynomial and dynamic power law regressions - checkout these links and my projects for more on this.

- Polynomial Regression Implementation: Github Repository
- Linear Regression Tutorial: Youtube Link
- Linear Regression Implementation: Github Repository

Thank you for reading this, I hope it helped!!